



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«МИРЭА – Российский технологический университет»
РТУ МИРЭА**

Институт Информационных Технологий

Кафедра Вычислительной техники

**ОТЧЕТ О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ
№5**

«Реализация автомата»

по дисциплине

«Теория формальных языков»

Выполнил студент группы ИКБО-43-23

Кощев М. И.

Принял ассистент

Цынгалёв П.С.

Практическая работа
выполнена

«14» ноября 2024 г.

«Зачтено»

«__» _____ 2024 г.

Москва 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ЦЕЛЬ	4
2 ЗАДАНИЕ	5
2.1 Формулировка задания	5
2.2 Математическая модель	5
2.3 Тестирование программы	6
3 ЗАКЛЮЧЕНИЕ	8
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	9
ПРИЛОЖЕНИЯ	10

ВВЕДЕНИЕ

Изучение теории формальных языков представляет собой значимый элемент образовательной программы по направлению программной инженерии. Эта область, являющаяся ключевой частью компьютерных наук и теоретической информатики, сосредотачивается на исследовании структуры и свойств формальных языков, а также на моделировании их обработки. В условиях современного общества, где возрастают объемы данных и потребность в эффективных методах их хранения, передачи и обработки, понимание основ формальных языков приобретает особую значимость.

Формальные языки находят применение в различных сферах: от описания синтаксиса языков программирования до разработки сетевых протоколов и создания компиляторов. С их помощью удастся формализовать взаимодействие как между человеком и компьютером, так и между компонентами программного обеспечения. В центре изучения теории формальных языков находятся такие понятия, как грамматики, автоматы и алгебраические структуры, которые позволяют описывать синтаксис и семантику языков.

В данном отчете мы рассмотрим преобразование выражения в обратную польскую запись.

1 ЦЕЛЬ

Графически представить недетерминированный конечный автомат, детерминированный конечный автомат и минимальный детерминированный конечный автомат (если возможно) согласно варианту. Реализовать автомат на выбранном ЯП согласно варианту.

2 ЗАДАНИЕ

2.1 Формулировка задания

На выбранном ЯП реализовать автомат согласно варианту.

Вариант: `letter(letter|digit|_)*`

2.2 Математическая модель

Создаём множества `letters` и `digits`, чтобы в дальнейшем классифицировать символы строки, как «буквы», «цифры» и «другие символы»

Разработаем метод, который создаёт автомат.

Для этого определяются множество состояний автомата: `states`, состоящее из чисел от 0 до 3, начальное состояние: `initial_state` – 0, множество конечных состояний `final_state`, состоящее из чисел от 1 до 3, словарь переходов, который определяет, как автомат переходит из одного состояния в другое в зависимости от входного символа – `transition_function`.

Реализуем функцию, которая принимает на вход строку, обрабатывает каждый её символ, определяя его тип (буква, цифра или символ подчёркивания), если символ не является допустимым, возвращает `false`. Проверяет наличие перехода для текущего состояния, если такой переход существует, автомат переходит в новое состояние, иначе возвращает `false`. В если в конце автомат оказался в одном из своих конечных состояний, то функция возвращает `true`

После обработки всех символов в строке, выводим сообщение: если строка соответствует автомату - “Строка принята”, иначе “Строка не принята”.

Графическое представление недетерминированного автомата продемонстрировано на рисунке 1.

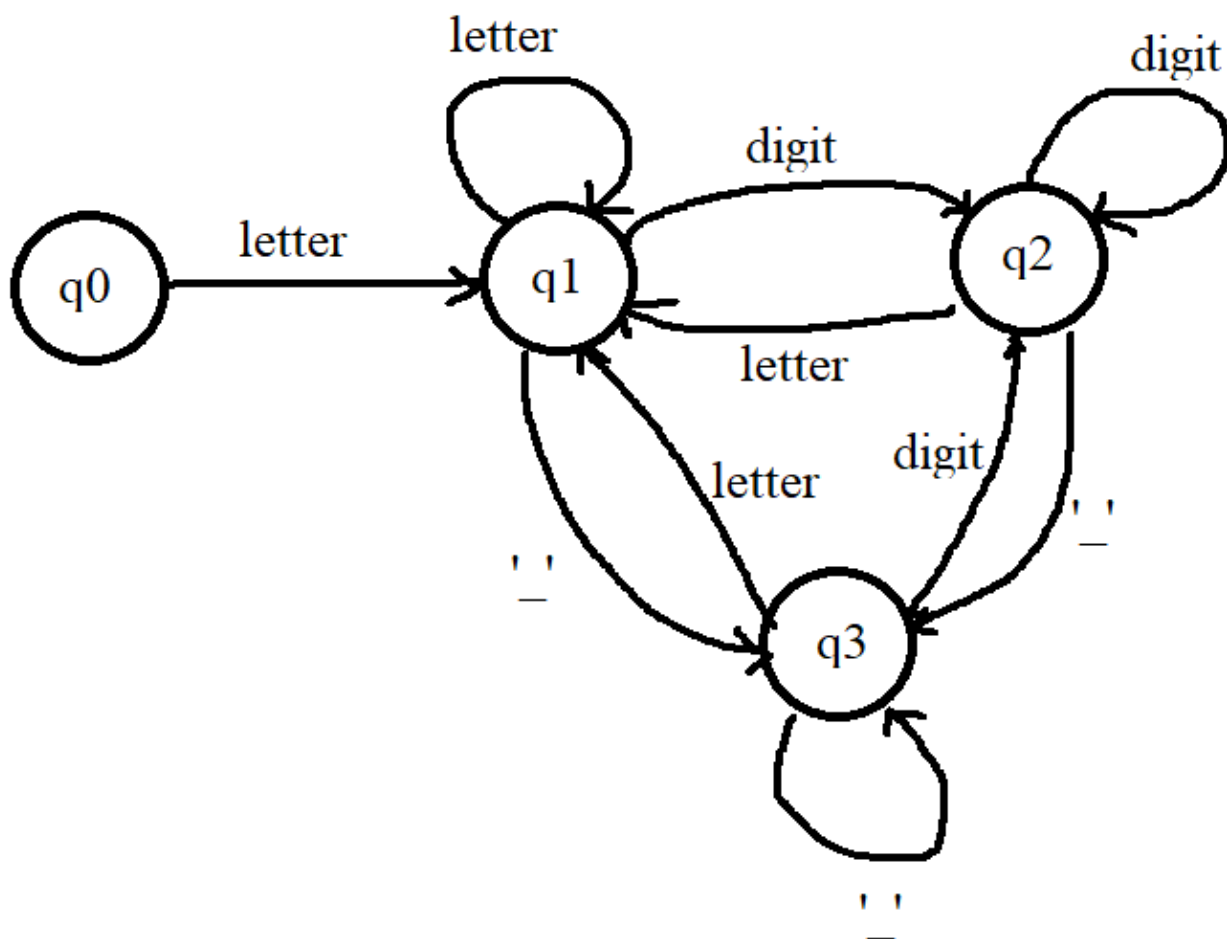


Рисунок 1 – Графическое представление недетерминированного автомата

Графическое представление детерминированного автомата продемонстрировано на рисунке 2.

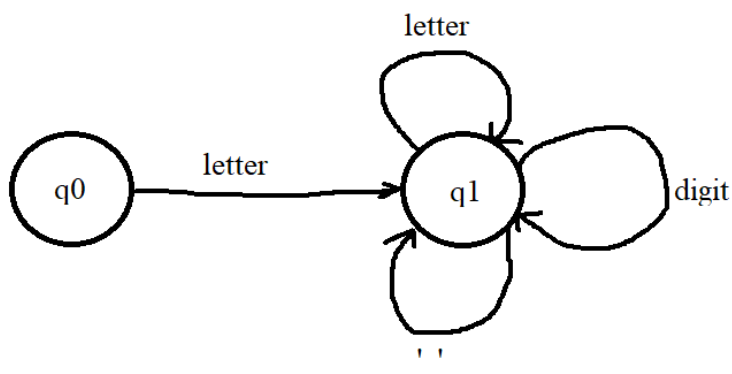


Рисунок 2 – Графическое представление детерминированного автомата

Графическое представление минимального детерминированного автомата продемонстрировано на рисунке 3, данный автомат будет аналогичен детерминированному конечному автомату.

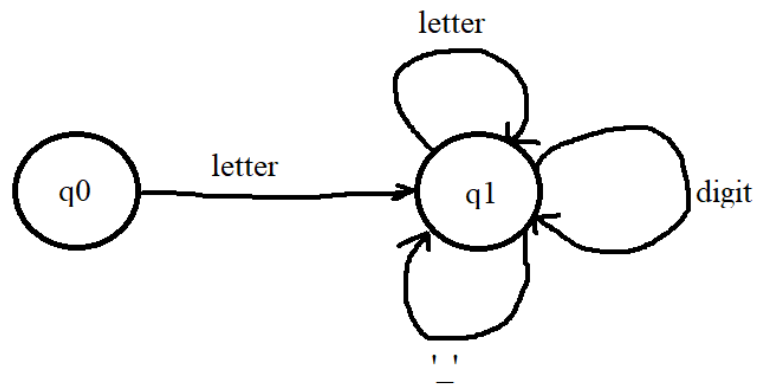
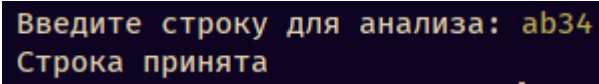


Рисунок 3 – Графическое представление минимального детерминированного автомата

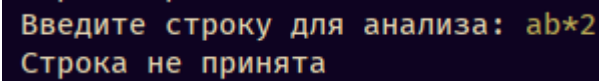
2.3 Тестирование программы

Проведём тестирование для данной программы на разных значениях. Результаты продемонстрируем на рисунке 1, 2.

A screenshot of a program's input/output window. It has a dark background with light-colored text. The first line says 'Введите строку для анализа: ab34' and the second line says 'Строка принята'.

Введите строку для анализа: ab34
Строка принята

Рисунок 4 - Первое тестирование программы

A screenshot of a program's input/output window, similar to the previous one. The first line says 'Введите строку для анализа: ab*2' and the second line says 'Строка не принята'.

Введите строку для анализа: ab*2
Строка не принята

Рисунок 5 - Второе тестирование программы

3 ЗАКЛЮЧЕНИЕ

В процессе выполнения практической работы были достигнуты следующие результаты:

Были реализованы: графическое представление недетерминированного конечного автомата, графическое представление детерминированного конечного автомата, графическое представление минимального детерминированного конечного автомата согласно варианту.

Реализован конечный автомат данного варианта на ЯП python.

Проведено тестирование программы, и результаты представлены в отчете.

Таким образом, поставленная цель работы — Графически представить недетерминированный конечный автомат, детерминированный конечный автомат и минимальный детерминированный конечный автомат (если возможно) согласно варианту. Реализовать автомат на выбранном ЯП согласно варианту — успешно достигнута.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Малявко А.А. Формальные языки и компиляторы: учебное пособие для вузов. - М., Изд-во Юрайт, 2019.
2. Свердлов С.З. Языки программирования и методы трансляции : учебное пособие для вузов - СПб., 2007, Id=65534.
3. Карпов Ю.Г. Теория и технология программирования. Основы построения трансляторов: учеб. пособие. - СПб.: БХВ-Петербург, 2005, Id=64347.

ПРИЛОЖЕНИЯ

Приложение А – код на языке Python

ПРИЛОЖЕНИЕ А

Код на языке Python

Листинг А.1 – main.py

```
from collections import defaultdict as dd

class FiniteStateMachine:
    def __init__(self):
        self.available_states = {0, 1, 2, 3}
        self.start_state = 0
        self.accepted_states = {1, 2, 3}
        self.current = self.start_state

        self.transitions = dd(lambda: None, {
            (0, 'alpha'): 1,
            (1, 'alpha'): 1,
            (1, 'num'): 2,
            (1, 'underscore'): 3,
            (2, 'num'): 2,
            (2, 'alpha'): 1,
            (2, 'underscore'): 3,
            (3, 'underscore'): 3,
            (3, 'alpha'): 1,
            (3, 'num'): 2,})

    def restart(self):
        self.current = self.start_state

    def classify_char(self, character: str) -> str:
        if character.isalpha():
            return 'alpha'
        elif character.isdigit():
            return 'num'
        elif character == '_':
            return 'underscore'
        return None

    def is_accepted(self) -> bool:
        return self.current in self.accepted_states

    def evaluate_input(self, input_seq: str) -> bool:
        self.restart()
```

Продолжение листинга A.1

```
        for char in input_seq:
            char_type = self.classify_char(char)
            if char_type is None:
                return False
            next_state = self.transitions[(self.current,
char_type)]
            if next_state is None:
                return False
            self.current = next_state
        else:
            return self.is_accepted()

if __name__ == "__main__":
    fsm = FiniteStateMachine()
    while True:
        user_input = input("Введите строку для анализа: ")
        if fsm.evaluate_input(user_input):
            print("Строка принята")
        else:
            print("Строка не принята")
```